Luật ghi Thomas

- Một cải tiến của thuật toán nhãn thời gian cơ bản TO
- Không bắt buộc phải là khả tuần tự xung đột nhưng từ chối vài hành động ghi.
- **Mục tiêu**: Giảm số lượng rollback không cần thiết khi thực hiện thao tác ghi.

Luật ghi Thomas

- Nguyên tắc hoạt động: Kiểm tra khi thực hiện Write(X)
  - Nếu Read_TS(X) > TS(T): T phải rollback (do có một giao tác mới hơn đã đọc X) và thao tác Write(X) bị từ chối.
  - Nếu Write_TS(X) > TS(T): không thực hiện Write(X), vẫn tiếp tục thực hiện T.
  - Nếu Read_TS(X) > TS(T) hoặc Write_TS(X) > TS(T) không xảy ra:
    - Thực hiện Write(X)
    - Cập nhật Write_TS(X) = TS(T)

Luật ghi Thomas

- Ví dụ: Giả sử: TS(T1) = 5, TS(T2) = 10
  - T2: Write(X) -> Write_TS(X) = 10
  - T1: Write(X)
  - Kiểm tra Write_TS(X) = 10 > TS(T1) = 5
  - Kết quả theo luật Thomas:
    - Bỏ qua Write(X) của T1
    - Không rollback T1.

Kỹ thuật xác nhận tính hợp lệ

- Ý tưởng chính:
  - Giả định xung đột giữa các giao tác xảy ra rất ít
  - Vì vậy, không kiểm tra khi giao tác đang chạy
  - Chỉ kiểm tra khi giao tác kết thúc

Kỹ thuật xác nhận tính hợp lệ

- Nguyên tắc hoạt động:
  - Trong quá trình thực hiện giao tác, các cập nhật không ghi trực tiếp vào CSDL
  - Thay vào đó các thay đổi được lưu trong bản sao dữ liệu cục bộ của giao tác.
  - Khi giao tác kết thúc hệ thống sẽ kiểm tra tính khả tuần tự
    - Nếu hợp lệ ghi dữ liệu vào CSDL
    - Nếu không hợp lệ giao tác rollback và chạy lại

Kỹ thuật xác nhận tính hợp lệ

- Ba giai đoạn chính:
  - **Giai đoạn đọc**: giao tác đọc dữ liệu từ CSDL.
    - Các Write không được ghi trực tiếp vào CSDL mà được lưu vào bản sao cục bộ (local copy)
  - **Giai đoạn xác nhận**: Kiểm tra xem giao tác có vi phạm serializability hay không.
    - Việc kiểm tra dựa trên ***read_set, write_set*** của giao tác. Nếu hợp lệ giao tác được chấp nhận, ngược lại giao tác bị abort .
  - **Giai đoạn ghi**:
    - Nếu giai đoạn xác nhận thành công, các cập nhật của giao tác được ghi vào CSDL. Ngược lại, các cập nhật bị loại bỏ và giao tác khởi động lại.

Kỹ thuật xác nhận tính hợp lệ

- Chi tiết của việc thực hiện kỹ thuật này:
  - Mỗi giao tác sẽ được lưu thông tin vào 2 tập ***read_set*** và ***write_set***
  - Khi xác nhận giao tác Ti, hệ thống kiểm tra với giao tác Tj chỉ cần đáp ứng 1 trong các điều kiện sau:
    - Tj hoàn thành Write trước khi Ti bắt đầu Read
    - Ti bắt đầu Write sau khi Tj hoàn thành Write và tập đọc ***read_set*** của Ti không có đơn vị dữ liệu nào trong ***write_set*** của Tj
    - ***read_set*** và ***write_set*** của Ti không có đơn vị dữ liệu nào trong write_set của Tj và Tj hoàn thành Read trước khi Ti hoàn thành Read

Kỹ thuật xác nhận tính hợp lệ

- Ưu điểm:
  - Không cần khóa dữ liệu
  - Không xảy ra deadlock
  - Hiệu suất cao khi xung đột ít
- Nhược điểm:
  - Khi nhiều giao tác chạy đồng thời thì nhiều giao tác sẽ rollback

  → Không hiệu quả trong môi trường xung đột cao

Các sơ đồ đa phiên bản

- Là kỹ thuật điều khiển tương tranh bằng cách lưu nhiều phiên bản của dữ liệu
- **Ý tưởng**: Khi dữ liệu cập nhật, hệ thống không ghi đè dữ liệu cũ mà tạo ra một phiên bản mới.
- Có 2 phương pháp chính:
  - Dựa trên giao thức nhãn thời gian
  - Dựa trên giao thức khóa 2 giai đoạn

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức nhãn thời gian

- Mỗi dữ liệu X có nhiều phiên bản: $X_1, X_2, ..., X_k$
- Mỗi Xi cần lưu thêm:
  - Read_TS(Xi): timestamp lớn nhất của giao tác đã đọc Xi
  - Write_TS(Xi): timestamp của giao tác đã ghi Xi

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức nhãn thời gian

- ***Khi giao tác T thực hiện Write(X):***
  - Hệ thống sẽ tạo phiên bản mới X(k+1)
  - Gán timestamp Write_TS(Xk+1) = TS(T), Read_TS(Xk+1) = TS(T)
  - Kết quả phiên bản cũ vẫn được giữ lại.

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức nhãn thời gian

- ***Khi giao tác T thực hiện Read(X)***:
  - Hệ thống sẽ tìm phiên bản Xi sao cho:
    - Write_TS(Xi) ≤ TS(T)
    - Và Write_TS(Xi) là lớn nhất.
  - Sau đó trả về giá trị Xi.
  - Đồng thời cập nhật Read_TS(Xi)

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức nhãn thời gian

- Để đảm bảo tính khả tuần tự cho lịch giao tác, 2 quy tắc sau được sử dụng:
  - Kiểm tra khi Write(X):
    - Nếu tồn tại 1 phiên bản Xi mà Write_TS(Xi) lớn nhất <=TS(T) và Read_TS(Xi)>TS(T) thì giao tác T bị hủy bỏ và Rollback.
    - Ngược lại, sẽ tạo ra 1 phiên bản mới Xj với nhãn thời gian của T

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức nhãn thời gian

- Để đảm bảo tính khả tuần tự cho lịch giao tác, 2 quy tắc sau được sử dụng:
  - Kiểm tra khi Read(X):
    - T tìm phiên bản Xi có Write_TS lớn nhất nhưng không vượt quá TS(T)
    - Sau khi đọc, nhãn Read_TS(Xi) được cập nhật.

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức nhãn thời gian

- Nhận xét:
  - Hành động đọc luôn thành công vì luôn có một phiên bản phù hợp để đọc.
  - Hành động ghi có thể bị rollback nếu Read_TS(Xi) > TS(T). Nguyên nhân là do một giao tác mới hơn đã đọc dữ liệu.
  - Nếu giao tác bị rollback các giao tác đọc dữ liệu từ nó cũng có thể phải rollback → Cascading Rollback

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức nhãn thời gian

- Ưu điểm:
  - Đọc không bị chặn
  - Tăng song song
  - Đảm bảo serializability
- Nhược điểm:
  - Tốn bộ nhớ lưu nhiều phiên bản
  - Cần garbage collection
  - Có thể xảy ra cascading rollback

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức khóa 2 giai đoạn

- Mỗi đơn vị dữ liệu X sẽ có ba chế độ khóa tương ứng với ba trạng thái:
  - Read_locked (Khóa đọc): Cho phép đọc dữ liệu.
  - Write_locked (Khóa ghi): Được cấp khi một giao tác muốn ghi dữ liệu lên một phiên bản mới.
  - Certify_locked (Khóa xác nhận): Được cấp khi giao tác sẵn sàng chấp nhận (commit) để chuyển đổi phiên bản mới thành phiên bản chính thức.

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức khóa 2 giai đoạn

- Mối quan hệ tương thích giữa khóa đọc và khóa ghi trong giao thức khóa chuẩn

	Read	**Write**
Read	Yes	No
Write	No	No

Các sơ đồ đa phiên bản

Kỹ thuật điều khiển đa phiên bản dựa trên giao thức khóa 2 giai đoạn

- Mối quan hệ tương thích giữa khóa đọc, khóa ghi và khóa xác nhận trong giao thức hai giai đoạn đa phiên bản

	Read	**Write**	**Certify**
Read	Yes	Yes	No
Write	Yes	No	No
Certify	No	No	No

Các sơ đồ đa phiên bản

Cơ chế hoạt động

- Kỹ thuật này duy trì hai phiên bản cho mỗi đơn vị dữ liệu X:
  - Phiên bản X: Là giá trị đã được ghi thành công bởi một giao tác đã chấp nhận (committed) trước đó.
  - Phiên bản X': Được tạo ra khi giao tác T yêu cầu khóa ghi. Giao tác T ghi giá trị lên X' mà không ảnh hưởng đến phiên bản X cũ.

Các sơ đồ đa phiên bản

Cơ chế hoạt động

- Quy trình xác nhận (Commit Process):
  - Khi T sẵn sàng chấp nhận, nó phải yêu cầu khóa xác nhận (Certify lock) trên tất cả đơn vị dữ liệu mà nó giữ khóa ghi.
  - Vì khóa xác nhận không tương thích với khóa đọc, T phải đợi cho đến khi các giao tác đang đọc giải phóng hết khóa trên đơn vị dữ liệu đó.
  - Sau khi có được khóa xác nhận (đây là khóa độc quyền), hệ thống sẽ cập nhật giá trị của X bằng X', loại bỏ X', và giải phóng khóa xác nhận.

Các sơ đồ đa phiên bản

- Ưu điểm:
  - Tăng hiệu suất: tối ưu truy cập đồng thời
  - Tránh hiện tượng Cascading Aborts
- Hạn chế:
  - Tốn bộ nhớ

Các giao thức khóa dựa trên đồ thị

- Giao thức khóa 2 giai đoạn đảm bảo tính khả tuần tự
  - nhưng cũng làm giảm hiệu suất truy cập đồng thời.
- Nếu không dùng thì cần thêm thông tin về thứ tự truy cập dữ liệu

➔Giao thức khóa dựa trên đồ thị

Các giao thức khóa dựa trên đồ thị

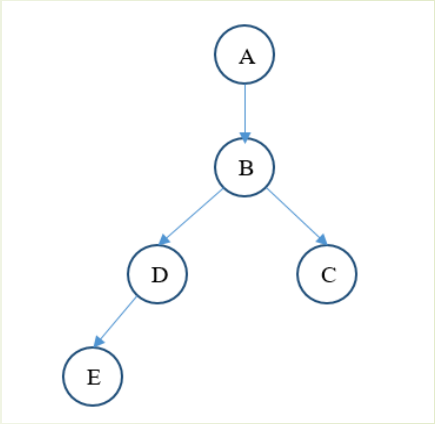
- Cách đơn giản nhất: Bổ sung thông tin thứ tự truy cập các đơn vị dữ liệu.
- Giả sử tập dữ liệu D = { d1, d2, d3, ..., dn }
  - di → dj: truy cập di trước rồi mới truy cập dj
  - Tập dữ liệu D tạo thành đồ thị có hướng không chu trình
- Đồ thị CSDL:
  - Mỗi đỉnh là một đơn vị dữ liệu
  - Mỗi cạnh biểu diễn thứ tự truy cập

Các giao thức khóa dựa trên đồ thị

- Quy tắc:
  - Khóa đầu tiên có thể trên bất kỳ dữ liệu nào.
  - Một dữ liệu Q chỉ được khóa nếu cha của Q đang bị khóa bởi T
  - Dữ liệu có thể Unlock bất kỳ lúc nào.
  - Nếu một dữ liệu đã Lock → Unlock thì không được lock lại lần nữa.

Các giao thức khóa dựa trên đồ thị

- Ví dụ: giả sử cây CSDL được biểu diễn như hình bên và 2 giao tác T1, T2 như sau:
  - T1: Lock(B); Lock(D); Lock(E); Unlock(B); Unlock(D); Unlock(E)
  - T2: Lock(B); Lock(C); Unlock(C); Unlock(B)
- Lịch giao tác tuân theo quy tắc của giao thức khóa dựa trên đồ thị



T1	T2
Lock(B)	
Lock(D)	
Lock(E)	
Unlock(B)	
	Lock(B)
	Lock(C)
	Unlock(C)
	Unlock(B)
Unlock(D)	
Unlock(E)	

Các giao thức khóa dựa trên đồ thị

- Ưu điểm:
  - Đảm bảo conflict serializability
  - Tránh deadlock
  - Cho phép unlock sớm
  - Tăng hiệu quả truy cập đồng thời
- Nhược điểm:
  - Cần biết cấu trúc truy cập dữ liệu trước
  - Khó áp dụng khi truy vấn phức tạp